P02 Better Jump

CRE132 worksheet

What you'll make

P01's jump works, but it feels stiff: every jump is the same height, falling is floaty, and the game is strict about when you may jump. Good platformers are generous instead. You'll make a new controller, `BetterJumpController`, from a copy of P01's, and add four small improvements one at a time:

- **Variable jump height:** let go of Jump early for a lower jump.
- **Faster fall:** stronger gravity on the way down.
- **Coyote time:** you can still jump for a moment after running off a ledge.
- **Jump buffering:** press Jump just before landing and you still jump when you land.

Each one has an "off" value in the Inspector, so you can switch it off during Play and feel the difference.

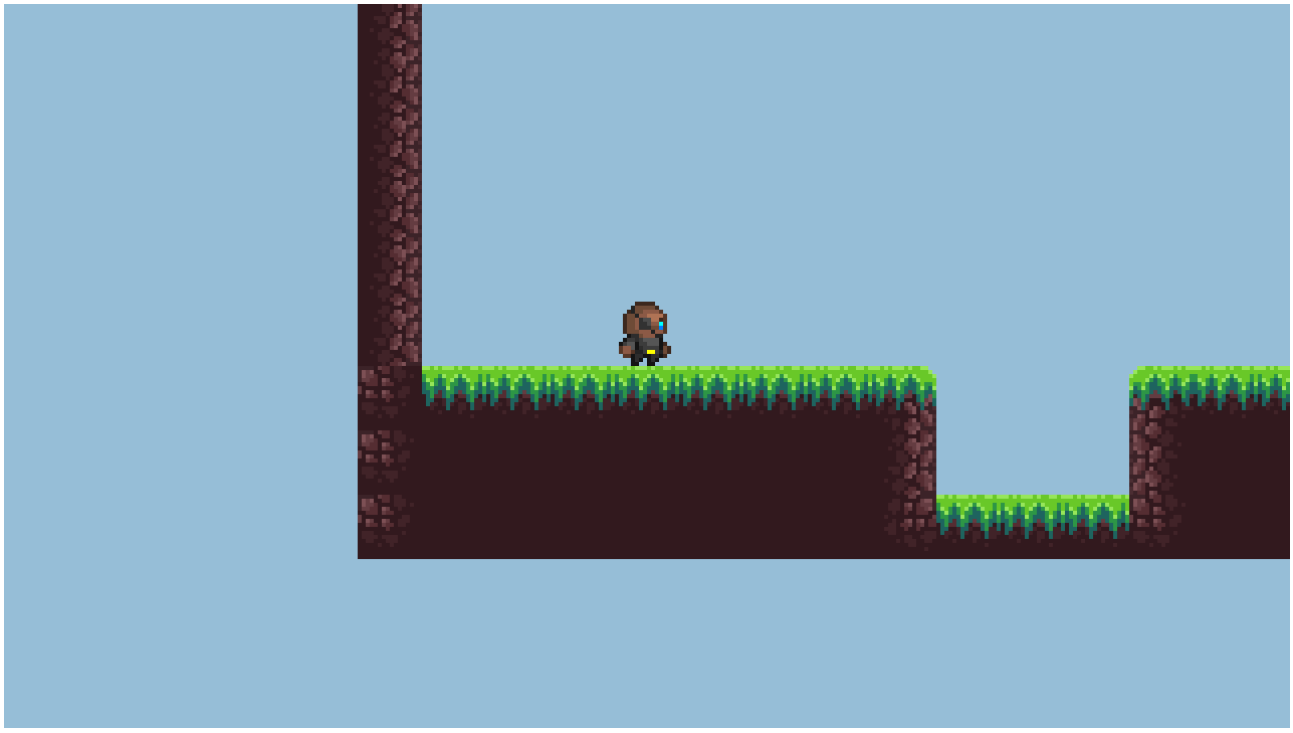


Figure 1: The start of the level: pits and ledges for testing jumps.

Before you start

1. Unzip `CRE132_P02_BetterJump_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_P02_BetterJump_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/Platformer/P02_BetterJump`.
3. Press **Play**. The player stands on the ground but can't move: it has no controller yet. Press Play again to stop.

You'll also need your `PlatformerController.cs` from P01. If yours doesn't work, use the one in `CRE132_P01_BasicMovement_Finished` (in `Assets/_game/Scripts/P01_BasicMovement`).

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Start from a copy: BetterJumpController

Make a folder `Assets/_game/Scripts/P02_BetterJump` and create a script in it called `BetterJumpController`. Double-click it to open it.

In your code editor, also open your P01 `PlatformerController.cs` (**File > Open File**, then find it in your P01 project folder). Select all of it, copy it, and paste it over everything in `BetterJumpController`. Then change the class line to `public class BetterJumpController : MonoBehaviour`, so it matches the file name. Change the comment at the top too, so it says what this script is.

Now add a property just below the private variables (below `private bool isGrounded;`):

```
    // Other scripts (the animator in P06) can ask whether the player is on the ground.
    public bool IsGrounded
    {
        get { return isGrounded; }
    }
```

- **Why a new file?** P01's controller stays as it was, so you can always go back and compare. In this course each sample's controller is its own script.
- **`IsGrounded`** has only a `get`, so other scripts can read whether the player is on the ground but can't change it. The animator in P06 will use it.

Add it: select the **Player** and **Add Component > Better Jump Controller**. Fill the slots as in P01: drag the **Ground Check** child into **Ground Check**, set **Ground Layers** to **Ground**, and pick **Player/Move** and **Player/Jump** for **Move Action** and **Jump Action**.

Test it: press Play. It should run and jump exactly like P01. Now you'll make the jump better, one change at a time.

2. Variable jump height

At the moment, a tap and a long press give the same jump. Add a field below `jumpSpeed`:

```
    // 1. Variable jump height: upward speed is multiplied by this when Jump is let go
    // (1 = off).
    [SerializeField] private float jumpCutMultiplier = 0.5f;
```

Add a new bool below `private bool jumpPressed;`:

```
    private bool jumpReleased;
```

In `Update`, below the `if` that checks `WasPressedThisFrame`, add:

```
        if (jumpAction.action.WasReleasedThisFrame())
        {
            jumpReleased = true;
        }
```

In `FixedUpdate`, below the line `jumpPressed = false;` and above `body.linearVelocity = velocity;`, add:

```
        // 1. Variable jump height: let go while still going up and the rest of the rise
        // is cut short.
```

```
        if (jumpReleased && velocity.y > 0f)
        {
            velocity.y = velocity.y * jumpCutMultiplier;
        }
        jumpReleased = false;
```

- **`WasReleasedThisFrame`** is the opposite of `WasPressedThisFrame`: true in the frame you let go. As with the press, `Update` notices it and `FixedUpdate` uses it.
- **`velocity.y > 0f`** means "still going up". Letting go on the way down does nothing.
- **Multiplying by 0.5** halves the upward speed, so the rest of the jump is much lower. A multiplier of 1 changes nothing, which switches the feature off.

Test it: press Play. Tap Space for a small hop, hold it for a full jump. Then set **Jump Cut Multiplier** to **1** and feel the difference. Set it back to **0.5**.

3. Faster fall

P01's jump goes up and comes down at the same rate, which feels floaty. Add these fields below `jumpCutMultiplier`:

```
    // 2. Faster fall: gravity going up and coming down.
    [SerializeField] private float normalGravityScale = 3f;
    [SerializeField] private float fallGravityScale = 5f;
    [SerializeField] private float maxFallSpeed = 18f;
```

In `FixedUpdate`, below the jump cut code you just added and above `body.linearVelocity = velocity;`, add:

```
        // 2. Faster fall: more gravity when moving down, and a top speed so falls don't
        // get out of hand.
        if (velocity.y < 0f)
        {
            body.gravityScale = fallGravityScale;
        }
        else
        {
            body.gravityScale = normalGravityScale;
        }
        if (velocity.y < -maxFallSpeed)
        {
            velocity.y = -maxFallSpeed;
        }
```

- **`body.gravityScale`** is the Rigidbody 2D's **Gravity Scale**. The script now sets it every physics step: 3 going up (the same as P01), 5 coming down.
- **`maxFallSpeed`** is a top speed for falling. Without it a long fall gets faster and faster until the player is hard to control.

Test it: jump and watch the fall: it's quicker than the rise. Set **Fall Gravity Scale** to **3** (the same as **Normal Gravity Scale**) to switch it off and compare. Set it back to **5**.

4. Coyote time

Run off a ledge and press Jump a moment too late: you fall, and it feels unfair. Coyote time is named after the cartoon coyote who runs off a cliff and only falls when he looks down. Add this field below `maxFallSpeed`:

```
    // 3. Coyote time and 4. jump buffering, in seconds (0 = off).
    [SerializeField] private float coyoteTime = 0.1f;
```

Add a counter below `private bool isGrounded;`:

```
    // Count down from coyoteTime / jumpBufferTime. Below zero means "no longer
    // allowed".
    private float coyoteCounter = -1f;
```

In `FixedUpdate`, below the ground check (the `if (ground != null)` and its `else`), add:

```
        // 3. Coyote time: full while on the ground, then it runs down after leaving it.
        if (isGrounded)
        {
            coyoteCounter = coyoteTime;
        }
        else
        {
            coyoteCounter = coyoteCounter - Time.fixedDeltaTime;
        }
```

Now change the jump itself. In the jump `if`, change `isGrounded` to `coyoteCounter >= 0f`, so it reads `if (jumpPressed && coyoteCounter >= 0f)`. Inside its braces, below `velocity.y = jumpSpeed;`, add the line `coyoteCounter = -1f;`.

- **A countdown timer:** while grounded, the counter is filled up to 0.1 seconds. In the air, `Time.fixedDeltaTime` (the time of one physics step) is taken off every step. While it's still 0 or more, you may jump.
- **Using it up:** after a jump, the counter is set to −1, which means "not allowed". Without that line, pressing Jump again straight after jumping would give you a second jump in the air.
- **`-1f` to start with** means the player can't jump before the first ground check has run.

Test it: run off the first ledge and press Jump a moment late: you still jump. Set **Coyote Time** to **0** and try again: now you fall. Set it back to **0.1**.

5. Jump buffering

Press Jump just before you land and nothing happens, because you weren't on the ground yet. Jump buffering remembers the press for a moment. Add this field below `coyoteTime`:

```
    [SerializeField] private float jumpBufferTime = 0.1f;
```

Add a second counter below `coyoteCounter`:

```
    private float jumpBufferCounter = -1f;
```

In `FixedUpdate`, below the coyote time code and above `Vector2 velocity = body.linearVelocity;`, add:

```
        // 4. Jump buffering: a press is remembered for jumpBufferTime, then forgotten.
        if (jumpPressed)
        {
            jumpBufferCounter = jumpBufferTime;
            jumpPressed = false;
        }
        else
```

```
        {
            jumpBufferCounter = jumpBufferCounter - Time.fixedDeltaTime;
        }
```

Finally, replace the jump. Delete the whole jump `if` (from `if (jumpPressed && coyoteCounter >= 0f)` to its closing brace) and the `jumpPressed = false;` line below it, and put this in their place:

```
        // Jump if Jump was pressed recently AND we were on the ground recently.
        if (jumpBufferCounter >= 0f && coyoteCounter >= 0f)
        {
            velocity.y = jumpSpeed;
            // Use both up, so one press can't jump twice.
            jumpBufferCounter = -1f;
            coyoteCounter = -1f;
        }
```

- **The same countdown pattern as coyote time:** a press fills `jumpBufferCounter` to 0.1 seconds, and it runs down every physics step after that.
- **`jumpPressed = false;` has moved** into the buffering code: the press is turned into the buffer counter straight away, so the jump only needs to look at the counter.
- **Both counters are used up** after a jump, so one press can't jump twice.

Test it: fall from a high ledge and press Jump just before you land: you jump as soon as you touch the ground. Set **Jump Buffer Time** to **0** and try again: the early press is lost. Set it back to **0.1**.

Check your work

- ☐ A tap gives a small hop and holding Jump gives a full jump.
- ☐ Falling is faster than rising.
- ☐ You can still jump just after running off a ledge.
- ☐ Pressing Jump just before landing jumps on landing.
- ☐ You can't jump a second time in the air.
- ☐ Compare with `CRE132_P02_BetterJump_Finished` if something behaves differently.

Key points

- **Good platformers are generous.** They help the player rather than following physics exactly. The player never notices coyote time or jump buffering, only that the game feels fair.
- **Countdown timers are a common pattern:** set a counter to a value, take `Time.fixedDeltaTime` off it every physics step, and check whether it's still 0 or more.
- **`IsGrounded` is a public read-only property**, so other scripts (the P06 animator) can ask about the player without being able to change it.

If something goes wrong

- **The script won't go on the Player ("no MonoBehaviour scripts in the file, or their names do not match"):** after pasting P01's code, the class line still says `PlatformerController`. Change it to `BetterJumpController`.
- **NullReferenceException:** a slot on the Better Jump Controller is empty: Ground Check, Move Action or Jump Action.
- **Changing the Rigidbody 2D's Gravity Scale does nothing:** that's expected now. The script sets it every step, so change **Normal Gravity Scale** and **Fall Gravity Scale** instead.
- **The player can jump again in the air:** a line that sets a counter to `-1f` after the jump is missing.
- **The jump never happens:** check you deleted the old jump `if` and pasted the new one inside `FixedUpdate`, between `Vector2 velocity = ...` and the jump cut code.

Try it

1. Raise **Coyote Time** to **0.5** and run off ledges. When does “generous” become “broken”?
2. Set **Jump Cut Multiplier** to **0** for an instant stop when you let go.
3. Add an *apex hang*: less gravity at the top of the jump, when `velocity.y` is close to 0 (for example between −1 and 1). Where in `FixedUpdate` would that go?